

DOCUMENTACIÓN TÉCNICA · .NET 8

VentasParalelo

Arquitectura por capas y recorrido del código: cómo está construido un motor de agregación paralela y por qué cada pieza está donde está.

Proyecto: motor de agregación analítica sobre transacciones de venta

Repositorio: github.com/IsraelDev-var/VentasParalelo

Contenido: 8 capas · 9 estrategias de agregación · 61 pruebas unitarias

Contenido

1. Visión general y mapa de capas

Qué hace el programa y cómo se conectan las piezas

2. Capa de modelos de dominio

SaleRecord y AggregationResult: los datos que viajan por todo el sistema

3. Capa de generación de datos

SalesDataGenerator: datasets sintéticos reproducibles

4. Capa de entrada/salida

SalesCsvReader: del archivo a un arreglo en memoria

5. Capa de particionado

Cómo se reparten las filas entre hilos, sin procesarlas todavía

6. Capa de estrategias de agregación

El núcleo del proyecto: 9 formas de resolver el mismo problema

7. Capa de métricas

AggregationDiagnostics y BenchmarkRunner: cómo se mide sin distorsionar

8. Capa de presentación (CLI)

Los tres comandos y el cálculo de speedup y eficiencia

9. Capa de pruebas

Por qué 61 pruebas y qué garantiza cada una

10. Cómo se mide, y cómo se midió mal

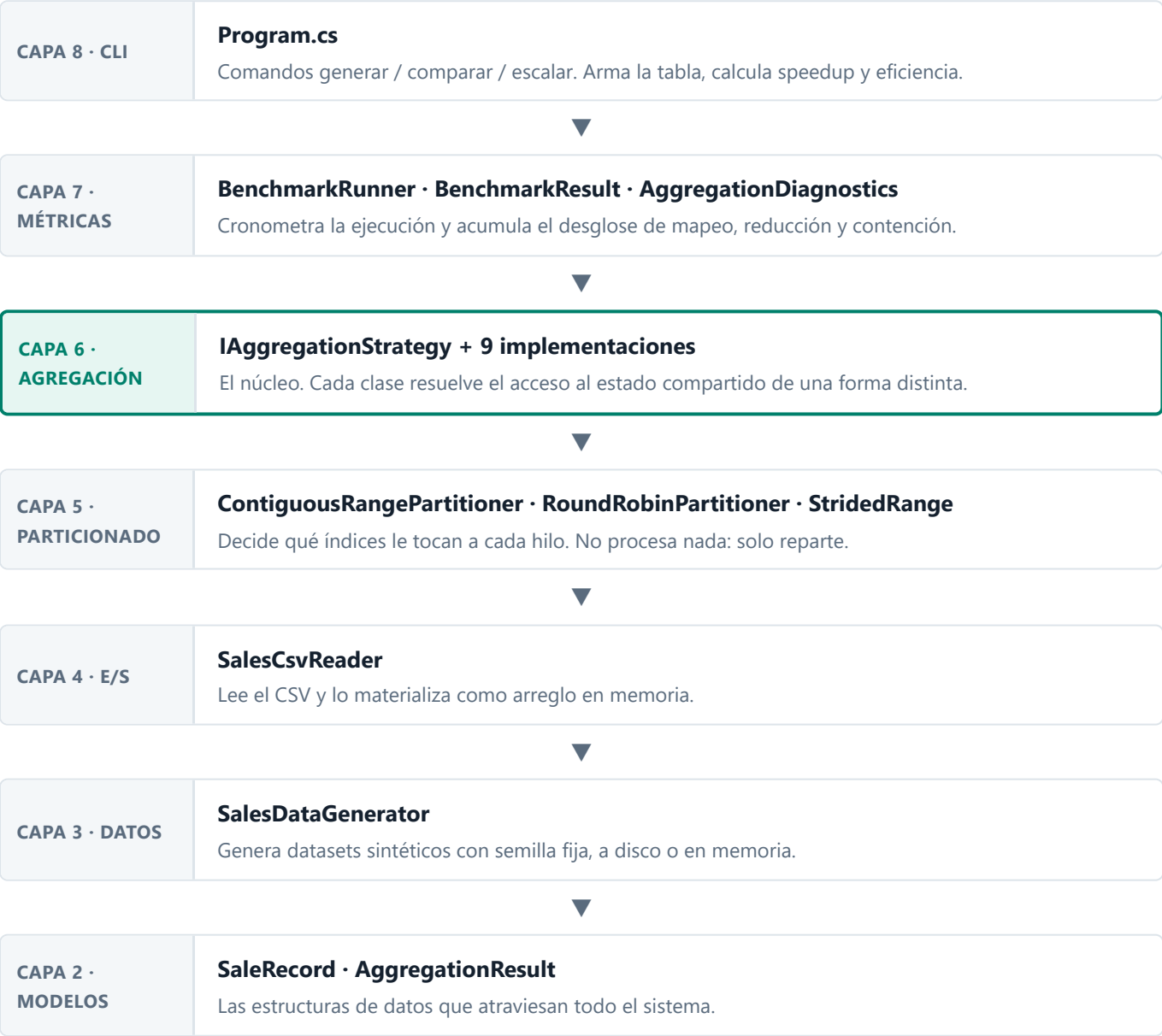
El error de medición que hubo que corregir, y la conclusión que se cayó

1. Visión general y mapa de capas

Qué hace el programa y cómo se conectan las piezas.

VentasParalelo resuelve un problema deliberadamente simple —sumar montos por sucursal y unidades por producto sobre millones de transacciones— para poder usarlo como banco de pruebas: como el cálculo es trivial, cualquier diferencia de rendimiento entre estrategias viene de **cómo se coordinan los hilos**, no del algoritmo de negocio.

La solución tiene tres proyectos: `VentasParalelo.Core` (toda la lógica), `VentasParalelo.Cli` (la interfaz de línea de comandos) y `VentasParalelo.Core.Tests` (las pruebas). Dentro de `Core`, el código está separado en capas donde cada una depende solo de las de abajo:



El flujo de una corrida completa

1 El CLI interpreta el comando

Por ejemplo `comparar --archivo data/ventas_5m.csv --hilos 1,2,4,8`.

2 Se carga el dataset una sola vez

`SalesCsvReader` devuelve un `SaleRecord[]`. Se reutiliza para todas las estrategias, así ninguna paga el costo de leer disco.

3 Se recorre la lista de estrategias

Para cada una y cada número de hilos, `BenchmarkRunner` cronometra una ejecución.

4 La estrategia pide su particionado

Recibe el arreglo completo y decide, con ayuda de la capa de particionado, qué índices procesa cada hilo.

5 Los hilos agregan

Aquí está toda la diferencia: unas estrategias sincronizan en cada fila, otras solo al terminar su partición.

6 Se verifica y se imprime

El resultado se compara contra el de la primera estrategia; si no coincide, la fila se marca con `(!)`. Luego se calculan `speedup` y `eficiencia`.

La regla de diseño que sostiene todo

Las nueve estrategias implementan la misma interfaz y comparten la misma función de acumulación. Lo único que cambia entre ellas es el particionado y la sincronización — por eso las comparaciones son justas: no hay una que tenga una ventaja algorítmica escondida.

2. Capa de modelos de dominio

Las estructuras de datos que viajan por todo el sistema.

2.1 SaleRecord — una transacción

src/VentasParalelo.Core/Models/SaleRecord.cs

```
public readonly record struct SaleRecord(  
    DateOnly Fecha,  
    string Sucursal,  
    string Producto,  
    int Cantidad,  
    decimal Monto);
```

Es un `readonly record struct`, y esa elección importa para el rendimiento. Al ser **struct** (tipo por valor), los cinco millones de registros viven dentro de un único arreglo contiguo en memoria, no como cinco millones de objetos dispersos en el heap. Eso significa que cuando un hilo recorre su partición, la CPU va cargando bloques de memoria consecutivos a la caché: es justamente lo que hace que el particionado por rangos contiguos gane frente a round-robin.

Al ser **readonly**, además, ningún hilo puede modificar un registro mientras otro lo lee — los datos de entrada son inmutables durante toda la agregación, así que no hace falta sincronizar el acceso a ellos. Toda la sincronización del proyecto es sobre los *acumuladores*, nunca sobre los datos de entrada.

2.2 AggregationResult — el acumulador

src/VentasParalelo.Core/Models/AggregationResult.cs

Los acumuladores

```
public sealed class AggregationResult  
{  
    public Dictionary<string, decimal> MontoPorSucursal { get; } = new();  
    public Dictionary<string, long> UnidadesPorProducto { get; } = new();  
    public long FilasProcesadas { get; set; }  
}
```

MergeFrom: la operación que hace posible el paralelismo

Esta es la pieza clave de todo el diseño. Permite que cada hilo trabaje aislado en su propia copia y que al final los resultados parciales se sumen sin perder información:

```

/// <summary>
/// Fusiona un acumulador local (de una particion) dentro de este resultado compartido.
/// Se usa una vez por particion (en LocalFinally), no una vez por fila.
/// </summary>
public void MergeFrom(AggregationResult otro)
{
    foreach (var (clave, valor) in otro.MontoPorSucursal)
        MontoPorSucursal[clave] = MontoPorSucursal.GetValueOrDefault(clave) + valor;

    foreach (var (clave, valor) in otro.UnidadesPorProducto)
        UnidadesPorProducto[clave] = UnidadesPorProducto.GetValueOrDefault(clave) + valor;

    FilasProcesadas += otro.FilasProcesadas;
}

```

La propiedad matemática que se aprovecha aquí es que la suma es **asociativa y conmutativa**: da igual en qué orden se sumen los subtotales, el resultado final es el mismo. Por eso es válido que cada hilo sume por su lado y luego se fusionen los parciales. Si la operación fuera, por ejemplo, "la primera venta de cada sucursal", este patrón no funcionaría.

TotalesCoinciden: el juez de correctitud

```

public bool TotalesCoinciden(AggregationResult otro)
{
    if (FilasProcesadas != otro.FilasProcesadas) return false;

    return DiccionesIguales(MontoPorSucursal, otro.MontoPorSucursal)
        && DiccionesIguales(UnidadesPorProducto, otro.UnidadesPorProducto);
}

```

Se usa en dos lugares: en las pruebas unitarias, y en vivo dentro del comando `comparar`, que marca con (!) cualquier estrategia cuyos totales se desvíen de la referencia. Una estrategia rápida pero incorrecta queda expuesta de inmediato.

3. Capa de generación de datos

Datasets sintéticos, reproducibles y de cualquier tamaño.

```
src/VentasParalelo.Core/DataGeneration/SalesDataGenerator.cs
```

Para medir con seriedad hace falta poder repetir exactamente la misma corrida. Por eso el generador recibe una **semilla fija**: con la misma semilla y el mismo número de filas, el dataset generado es byte por byte idéntico, en esta máquina y en cualquier otra.

```
Semilla fija = resultados reproducibles

public static void GenerateCsv(string path, long rowCount, int seed = 12345)
{
    var random = new Random(seed);
    var startDate = new DateOnly(2025, 1, 1);

    using var writer = new StreamWriter(path, false, Encoding.UTF8);
    writer.WriteLine("fecha,sucursal,producto,cantidad,monto");
```

El catálogo es pequeño a propósito — 8 sucursales y 12 productos:

```
private static readonly string[] Sucursales =
[
    "Santo Domingo Centro", "Santiago", "La Vega", "San Cristobal",
    "Punta Cana", "Puerto Plata", "San Pedro de Macoris", "Bavaro"
];
```

Por qué un catálogo chico cambia el experimento

Con solo 8 sucursales, los diccionarios acumuladores tienen 8 claves, no millones. Eso hace que la operación de fusión sea baratísima y que *toda* la diferencia de rendimiento entre estrategias venga de la sincronización, no del costo de fusionar. También es lo que hace tan brutal el efecto del lock global: los 8 hilos pelean por las mismas 8 casillas de memoria.

Dos formas de generar

El generador expone dos caminos que comparten la misma lógica de fila (`GenerarRegistro`), para que los datos sean equivalentes:

Método	Destino	Para qué se usa
<code>GenerateCsv</code>	Archivo en disco	Comando <code>generar</code> . El dataset queda guardado y se reutiliza entre corridas.
<code>GenerateInMemory</code>	Arreglo en RAM	Comando <code>escalar</code> , que prueba muchos volúmenes seguidos: evita escribir y leer disco 20 millones de filas.

La lógica de fila, compartida por ambos caminos

```
private static SaleRecord GenerarRegistro(Random random, DateOnly startDate)
{
    const int dateRangeDays = 365;

    var fecha = startDate.AddDays(random.Next(dateRangeDays));
    var sucursal = Sucursales[random.Next(Sucursales.Length)];
    var producto = Productos[random.Next(Productos.Length)];
    var cantidad = random.Next(1, 21);
    var precioUnitario = 500 + random.NextDouble() * 45000;
    var monto = Math.Round((decimal)(cantidad * precioUnitario), 2);

    return new SaleRecord(fecha, sucursal, producto, cantidad, monto);
}
```

4. Capa de entrada/salida

Del archivo CSV a un arreglo listo para particionar.

src/VentasParalelo.Core/IO/SalesCsvReader.cs

El lector carga el archivo completo a memoria antes de que empiece cualquier medición. Esto es deliberado: si la lectura ocurriera durante la agregación, se estaría midiendo la velocidad del disco en lugar de la del paralelismo.

```
public static SaleRecord[] ReadAll(string path)
{
    var records = new List<SaleRecord>();
    using var reader = new StreamReader(path);

    reader.ReadLine(); // encabezado
    string? line;
    while ((line = reader.ReadLine()) != null)
    {
        if (line.Length == 0) continue;
        records.Add(ParseLine(line));
    }

    return [.. records];
}
```

El parseo, optimizado con Span

Parsear cinco millones de líneas es costoso, así que el proyecto evita crear objetos intermedios innecesarios usando `ReadOnlySpan<char>` — una "ventana" sobre la cadena original que no copia memoria:

```
public static SaleRecord ParseLine(ReadOnlySpan<char> line)
{
    Span<Range> fieldRanges = stackalloc Range[5];
    var fieldCount = line.Split(fieldRanges, ',');

    if (fieldCount != 5)
        throw new FormatException($"Se esperaban 5 campos, se encontraron {fieldCount}: {line}");

    var fecha = DateOnly.ParseExact(line[fieldRanges[0]], "yyyy-MM-dd", CultureInfo.InvariantCulture);
    var sucursal = line[fieldRanges[1]].ToString();
    var producto = line[fieldRanges[2]].ToString();
    var cantidad = int.Parse(line[fieldRanges[3]], CultureInfo.InvariantCulture);
    var monto = decimal.Parse(line[fieldRanges[4]], CultureInfo.InvariantCulture);

    return new SaleRecord(fecha, sucursal, producto, cantidad, monto);
}
```

`stackalloc` reserva el arreglo de cinco rangos en el **stack** en vez del heap. Como el stack de cada hilo es privado y se libera solo al salir del método, esto no genera basura que el recolector tenga que limpiar después — un detalle que importa cuando el método se llama millones de veces.

Limitación asumida conscientemente

Se usa un `Split` simple en vez de un parser CSV completo. Es válido solo porque los campos generados nunca contienen comas ni comillas. Con datos reales de un sistema de ventas haría falta un parser que respete el estándar CSV.

5. Capa de particionado

Cómo se reparten los índices entre hilos, sin procesar nada todavía.

Esta capa está separada del resto a propósito. Al aislar "cómo se reparte" de "cómo se acumula", el proyecto puede comparar dos particionados distintos ejecutando exactamente el mismo código de agregación — y así atribuir la diferencia de rendimiento únicamente al reparto.

5.1 Particionado contiguo

src/VentasParalelo.Core/Partitioning/ContiguousRangePartitioner.cs

Divide el rango `[0, length)` en bloques consecutivos del mismo tamaño. Cuando la división no es exacta, las primeras particiones reciben una fila extra, de modo que la diferencia máxima entre la partición más grande y la más chica es de una sola fila:

```
public static IReadOnlyList<Range> Partition(int length, int partitionCount)
{
    if (partitionCount < 1)
        throw new ArgumentOutOfRangeException(nameof(partitionCount));

    partitionCount = Math.Min(partitionCount, Math.Max(length, 1));

    var particiones = new List<Range>(partitionCount);
    var baseSize = length / partitionCount;
    var remainder = length % partitionCount;
    var start = 0;

    for (var i = 0; i < partitionCount; i++)
    {
        var size = baseSize + (i < remainder ? 1 : 0);
        var end = start + size;
        particiones.Add(new Range(start, end));
        start = end;
    }

    return particiones;
}
```

Con 1.000 filas y 4 hilos, el reparto queda: `[0,250)` `[250,500)` `[500,750)` `[750,1000)`. Cada hilo recorre memoria consecutiva, que es el patrón de acceso que la caché de la CPU premia.

5.2 Particionado round-robin

src/VentasParalelo.Core/Partitioning/RoundRobinPartitioner.cs · StridedRange.cs

Reparte las filas de forma alternada: el hilo `p` recibe los índices `p`, `p+P`, `p+2P`, Con 1.000 filas y 4 hilos, el hilo 0 procesa las filas 0, 4, 8, 12... Cada hilo recibe la misma cantidad de filas que en el caso contiguo, pero las va tomando salteadas.

StridedRange genera los índices sin materializar un arreglo

```
public readonly record struct StridedRange(int Start, int Step, int Count)
{
    public IEnumerable<int> Indices()
    {
        var indice = Start;
        for (var i = 0; i < Count; i++)
        {
            yield return indice;
            indice += Step;
        }
    }
}
```

El uso de `yield return` hace que los índices se generen sobre la marcha: no se reserva memoria para guardar la lista completa de índices de cada partición, que en un dataset de 5 millones de filas sería un desperdicio considerable.

Para qué existe una estrategia que se sabe peor

Round-robin reparte la carga igual de bien que el particionado contiguo — misma cantidad de filas por hilo — pero rinde unos 10 puntos porcentuales menos que las otras cuatro variantes del mismo patrón. Como el algoritmo de acumulación es idéntico, esa diferencia aísla y demuestra un efecto puro de **localidad de caché**: saltar por la memoria cuesta más que recorrerla en orden. Está en el proyecto justamente para poder medir eso.

6. Capa de estrategias de agregación

El núcleo del proyecto: nueve formas de resolver el mismo problema.

6.1 El contrato

src/VentasParalelo.Core/Aggregation/IAggregationStrategy.cs

```
public interface IAggregationStrategy
{
    string Nombre { get; }

    AggregationResult Aggregate(SaleRecord[] registros, int maxDegreeOfParallelism);
}
```

Una interfaz de dos miembros. Todo lo demás —particionar, sincronizar, fusionar— es decisión de cada implementación. Esto es lo que permite que el CLI las trate a todas por igual y que agregar una décima estrategia no requiera tocar ninguna otra parte del sistema.

6.2 El corazón compartido: Acumular

src/VentasParalelo.Core/Aggregation/SequentialAggregator.cs

Todas las estrategias (salvo las dos que delegan en estructuras de .NET) llaman a este mismo método estático. Es la garantía de que están haciendo idéntico trabajo por fila:

```
internal static void Acumular(AggregationResult result, in SaleRecord r)
{
    result.MontoPorSucursal[r.Sucursal] =
        result.MontoPorSucursal.GetValueOrDefault(r.Sucursal) + r.Monto;

    result.UnidadesPorProducto[r.Producto] =
        result.UnidadesPorProducto.GetValueOrDefault(r.Producto) + r.Cantidad;
}
```

Nótese que `Acumular` **no tiene ninguna sincronización**. Es responsabilidad de quien lo llama garantizar que no haya dos hilos escribiendo el mismo `AggregationResult` a la vez. Ahí es donde las nueve estrategias divergen.

6.3 Familia A — sincronizan en cada fila

LockDictionaryAggregator: el caso malo, a propósito

src/VentasParalelo.Core/Aggregation/LockDictionaryAggregator.cs

```
Parallel.ForEach(
    particiones,
    new ParallelOptions { MaxDegreeOfParallelism = maxDegreeOfParallelism },
    rango =>
    {
        for (var i = rango.Start.Value; i < rango.End.Value; i++)
        {
            var r = registros[i];

            // Tiempo de espera para adquirir el lock global: aproxima la contención
            // real que paga esta estrategia por sincronizar en cada fila.
            var espera = Stopwatch.StartNew();
            Monitor.Enter(sync);
            espera.Stop();
            diag.SumarContencion(espera.Elapsed);

            try
            {
                SequentialAggregator.Acumular(result, in r);
            }
            finally
            {
                Monitor.Exit(sync);
            }
        }
    });
```

Los hilos se reparten bien el trabajo, pero todos escriben en el *mismo* `AggregationResult`, protegido por un único candado. El resultado es correcto, pero el paralelismo se degrada a ejecución en serie **más** el costo de coordinarse.

El detalle interesante es cómo se mide la contención: en vez de usar `lock` (azúcar sintáctico), se usa `Monitor.Enter` explícito para poder cronometrar exactamente cuánto se tardó en *conseguir* el candado. Ese tiempo es contención pura — tiempo en el que el hilo está vivo pero sin hacer trabajo útil.

ConcurrentDictionaryAggregator: candados más finos

src/VentasParalelo.Core/Aggregation/ConcurrentDictionaryAggregator.cs

```
for (var i = rango.Start.Value; i < rango.End.Value; i++)
{
    var r = registros[i];
    montoPorSucursal.AddOrUpdate(r.Sucursal, r.Monto, (_, actual) => actual + r.Monto);
    unidadesPorProducto.AddOrUpdate(r.Producto, r.Cantidad, (_, actual) => actual + r.Cantidad);
}
```

En lugar de un candado global, `ConcurrentDictionary` usa candados internos por "bucket". Reduce la contención, pero sigue sincronizando una vez por fila. Además, esa sincronización es interna y opaca: no hay forma de medirla desde afuera, por eso esta estrategia no reporta diagnósticos.

PlinqGroupByAggregator: el nivel más alto de abstracción

src/VentasParalelo.Core/Aggregation/PlinqGroupByAggregator.cs

```
var montoPorSucursal = registros
    .AsParallel()
    .WithDegreeOfParallelism(grado)
    .GroupBy(r => r.Sucursal)
    .Select(g => (Clave: g.Key, Total: g.Sum(r => r.Monto)))
    .ToArray();
```

El código más corto y legible de todo el proyecto — y el que peor rinde. PLINQ decide internamente cómo particionar y fusionar; a cambio, no se puede controlar ni instrumentar. Además recorre el arreglo **dos veces** (una por cada agrupación), a diferencia de las otras estrategias que hacen ambos acumuladores en una sola pasada.

6.4 Familia B — sincronizan una vez por partición

LocalReductionAggregator: el patrón map-reduce

```
src/VentasParalelo.Core/Aggregation/LocalReductionAggregator.cs
```

Aquí está la idea central del proyecto, implementada con el mecanismo `localInit` / `localFinally` de `Parallel.ForEach`:

```
Parallel.ForEach(
    particiones,
    new ParallelOptions { MaxDegreeOfParallelism = maxDegreeOfParallelism },
    localInit: () => new AggregationResult(),
    body: (rango, _, local) =>
    {
        var mapeo = Stopwatch.StartNew();
        for (var i = rango.Start.Value; i < rango.End.Value; i++)
            SequentialAggregator.Acumular(local, i, registros[i]);

        local.FilasProcesadas += rango.End.Value - rango.Start.Value;
        mapeo.Stop();
        diag.SumarMapeo(mapeo.Elapsed);
        return local;
    },
    localFinally: local =>
    {
        // El merge solo ocurre una vez por particion (no una vez por fila), asi que
        // el lock aqui casi no compite: por eso no se mide como "contencion" aparte,
        // sino como parte del costo fijo de la reduccion.
        var reduccion = Stopwatch.StartNew();
        lock (mergeLock)
        {
            resultado.MergeFrom(local);
        }
        reduccion.Stop();
        diag.SumarReduccion(reduccion.Elapsed);
    });
```

El mecanismo tiene tres partes:

Parte	Cuándo se ejecuta	Qué hace
<code>localInit</code>	Una vez por hilo, al empezar	Crea un acumulador privado. Nadie más lo toca.
<code>body</code>	Una vez por partición	Procesa las filas contra el acumulador privado. Sin ningún lock.
<code>localFinally</code>	Una vez por hilo, al terminar	Toma el lock una sola vez y fusiona su acumulador en el resultado compartido.

El cambio de escala que lo explica todo

Con 5 millones de filas y 8 hilos, `LockDictionaryAggregator` toma el candado 5.000.000 de veces.

`LocalReductionAggregator` lo toma 8 veces. El trabajo útil es idéntico; lo que cambia es el número de puntos de sincronización, y con él, la eficiencia: de un 6 % a un 41 %.

Las tres variantes del mismo patrón

Estas comparten el algoritmo de `LocalReductionAggregator` y cambian un solo aspecto, para poder medir su efecto por separado:

Estrategia	Qué cambia respecto al patrón base
<code>RoundRobinReductionAggregator</code>	Solo el particionado: usa <code>RoundRobinPartitioner</code> en lugar de rangos contiguos. Aísla el efecto de la localidad de caché.
<code>DynamicChunkReductionAggregator</code>	Reparte trozos pequeños (4.096 filas) de forma dinámica desde una cola compartida, en vez de fijar las particiones de antemano. Balancea mejor si el costo por fila fuera irregular.
<code>HierarchicalReductionAggregator</code>	Solo la fusión final: combina los parciales de a pares en un árbol binario en paralelo, en vez de un lock secuencial. Pasa de $O(P)$ a $O(\log P)$ niveles.

Chunking dinámico: el reparto lo decide el runtime, no el programador

```
var chunkSize = Math.Max(1, Math.Min(_tamanoChunk, registros.Length));
var particionador = Partitioner.Create(0, registros.Length, chunkSize);
```

Reducción en árbol: divide y vencerás sobre los resultados parciales

```
private static AggregationResult ReducirEnArbol(
    AggregationResult[] locales, int inicio, int fin, AggregationDiagnostics diag)
{
    var cantidad = fin - inicio;
    if (cantidad == 1)
        return locales[inicio];

    var medio = inicio + cantidad / 2;

    AggregationResult izquierda = null!;
    AggregationResult derecha = null!;
    Parallel.Invoke(
        () => izquierda = ReducirEnArbol(locales, inicio, medio, diag),
        () => derecha = ReducirEnArbol(locales, medio, fin, diag));

    var reduccion = Stopwatch.StartNew();
    izquierda.MergeFrom(derecha);
    reduccion.Stop();
    diag.SumarReduccion(reduccion.Elapsed);

    return izquierda;
}
```

CoarseGrainedTaskAggregator: cero estado compartido

src/VentasParalelo.Core/Aggregation/CoarseGrainedTaskAggregator.cs

Lleva la idea al extremo. Cada hilo escribe su resultado en **su propio slot de un arreglo**, así que durante todo el cómputo no existe ni un solo punto de sincronización. La fusión ocurre después, en un único hilo, cuando ya no hay escritura concurrente posible:

```

var locales = new AggregationResult[particiones.Count];
var diag = new AggregationDiagnostics();

Parallel.For(
    0, particiones.Count,
    new ParallelOptions { MaxDegreeOfParallelism = maxDegreeOfParallelism },
    i =>
    {
        var rango = particiones[i];
        var local = new AggregationResult();
        var mapeo = Stopwatch.StartNew();

        for (var j = rango.Start.Value; j < rango.End.Value; j++)
            SequentialAggregator.Acumular(local, in registros[j]);

        local.FilasProcesadas = rango.End.Value - rango.Start.Value;
        mapeo.Stop();
        diag.SumarMapeo(mapeo.Elapsed);
        locales[i] = local;
    });

var reduccion = Stopwatch.StartNew();
var resultado = new AggregationResult();
foreach (var local in locales)
    resultado.MergeFrom(local);
reduccion.Stop();
diag.SumarReduccion(reduccion.Elapsed);

resultado.Diagnostics = diag;

```

Escribir en `locales[i]` desde varios hilos es seguro sin ningún lock porque cada hilo escribe en un índice distinto: no hay dos hilos tocando la misma posición de memoria.

Lo que la medición reveló sobre esta estrategia

La hipótesis era que eliminar hasta el último punto de sincronización la haría la más rápida. Al promediar tres corridas resultó estar empatada con acumuladores locales y reducción en árbol (38–41 % de eficiencia). La conclusión: una vez eliminada la sincronización por fila, el techo lo pone la cantidad de núcleos físicos, no el detalle fino del diseño.

7. Capa de métricas

Cómo se mide sin que la medición distorsione lo medido.

7.1 AggregationDiagnostics

```
src/VentasParalelo.Core/Metrics/AggregationDiagnostics.cs
```

Este acumulador recibe tiempos **desde varios hilos a la vez**. Si usara un `long` normal con `+=`, dos hilos podrían leer el mismo valor y pisarse mutuamente, perdiendo mediciones. La solución no es un lock —que agregaría justamente la contención que se quiere medir— sino operaciones atómicas:

```
public sealed class AggregationDiagnostics
{
    private long _tiempoMapeoTicks;
    private long _tiempoReduccionTicks;
    private long _tiempoContencionTicks;

    public TimeSpan TiempoMapeo => TimeSpan.FromTicks(Interlocked.Read(ref _tiempoMapeoTicks));
    public TimeSpan TiempoReduccion => TimeSpan.FromTicks(Interlocked.Read(ref _tiempoReduccionTicks));
    public TimeSpan TiempoContencion => TimeSpan.FromTicks(Interlocked.Read(ref
    _tiempoContencionTicks));

    public void SumarMapeo(TimeSpan transcurrido) => Interlocked.Add(ref _tiempoMapeoTicks,
    transcurrido.Ticks);
    public void SumarReduccion(TimeSpan transcurrido) => Interlocked.Add(ref _tiempoReduccionTicks,
    transcurrido.Ticks);
    public void SumarContencion(TimeSpan transcurrido) => Interlocked.Add(ref _tiempoContencionTicks,
    transcurrido.Ticks);
}
```

`Interlocked.Add` ejecuta "leer, sumar y escribir" como una única instrucción indivisible del procesador. Es correcta bajo concurrencia y mucho más barata que un lock.

Métrica	Qué mide	Qué estrategias la reportan
Mapeo	Tiempo procesando filas, sin sincronizar. Trabajo útil.	Las cinco de la familia B
Reducción	Tiempo fusionando resultados parciales.	Las cinco de la familia B
Contención	Tiempo esperando para adquirir un candado. Tiempo perdido.	Solo lock sobre Dictionary

Por qué la suma de tiempos supera el tiempo total

Estas cifras son la *suma de todos los hilos*, no tiempo de reloj. Con 8 hilos corriendo 0,5 s cada uno, el mapeo reporta 4 s aunque en el reloj hayan pasado 0,5 s. Por eso la contención del lock llega a 6,41 s en una corrida que duró menos de un segundo: son ocho hilos esperando simultáneamente.

7.2 BenchmarkRunner

```
src/VentasParalelo.Core/Metrics/BenchmarkRunner.cs
```

```
public const int CalentamientosPorDefecto = 1;
public const int RepeticionesPorDefecto = 3;

/// <summary>
/// Cronometra una estrategia descartando primero una o mas ejecuciones de calentamiento y
/// quedandose despues con el MEJOR tiempo de varias repeticiones.
/// </summary>
/// <remarks>
/// El calentamiento no es un lujo: La primera ejecucion de un proceso paga la compilacion
/// JIT, el crecimiento inicial de Los diccionarios y las caches frias, y midio ~50% mas
```

Deliberadamente mínimo. El cronómetro arranca justo antes de `Aggregate` y para justo después: no incluye la carga del archivo ni la impresión de resultados. Lo único que se mide es la agregación.

8. Capa de presentación (CLI)

Los tres comandos y el cálculo de las métricas comparativas.

```
src/VentasParalelo.Cli/Program.cs
```

El punto de entrada usa *top-level statements* de C# y despacha a tres comandos:

```
return args[0] switch
{
    "generar" => Generar(args),
    "comparar" => Comparar(args),
    "escalar" => Escalar(args),
    _ => Fallback()
};
```

Comando	Qué hace
generar	Crea un dataset CSV del tamaño pedido.
comparar	Corre las 9 estrategias sobre el mismo dataset con distintos números de hilos y arma la tabla comparativa.
escalar	Barrido de escalabilidad fuerte (mismo dataset, más hilos) o débil (datos y hilos crecen juntos).

8.1 El cálculo de speedup y eficiencia

Estas dos líneas son el resultado que todo el proyecto persigue:

```
// inflaba el speedup de todas las demas. Calentando todas antes de medir ninguna, el orden
// dentro de la tabla deja de influir en el resultado.
Console.WriteLine("Calentando las estrategias antes de medir...");
foreach (var estrategiaCalentamiento in estrategias)
    estrategiaCalentamiento.Aggregate(registros, Math.Max(1, hilos.Max()));
Console.WriteLine();

Console.WriteLine($"{ "Estrategia",-52}{ "Hilos",6}{ "Tiempo (s)",12}{ "Filas/seg",14}{ "Speedup",10}
{"Eficiencia",11}");
Console.WriteLine(new string('-', 105));
```

Métrica	Fórmula	Cómo se lee
Speedup	tiempo base ÷ tiempo actual	Cuántas veces más rápido. 8,00 con 8 hilos sería perfecto; menos de 1,00 significa que se volvió más lento.
Eficiencia	speedup ÷ número de hilos	Qué proporción del paralelismo disponible se aprovechó. 100 % es el ideal teórico.

El `??=` en `baselineSegundos ??= ...` asigna solo si el valor es nulo: el baseline se fija con la primera medición (el agregador secuencial) y todas las demás se comparan contra él.

La verificación que corre en cada fila de la tabla

Antes de imprimir cada resultado, el CLI compara los totales contra la primera estrategia ejecutada. Si difieren, agrega (!) totales distintos a la referencia. Así, una estrategia con una condición de carrera no puede aparecer como "rápida" sin que salte la alarma.

```
Console.WriteLine($"{ "Estrategia",-52}{ "Hilos",6}{ "Tiempo (s)",12}{ "Filas/seg",14}{ "Speedup",10}{  
"Eficiencia",11}");
```

9. Capa de pruebas

61 pruebas que garantizan que la comparación tiene sentido.

```
tests/VentasParalelo.Core.Tests/
```

En un proyecto de rendimiento, las pruebas cumplen un papel particular: no solo verifican que el código funcione, sino que **legitiman la comparación**. Una estrategia que va rapidísimo pero pierde filas no es más rápida, es incorrecta.

9.1 La prueba central

```
[Theory]
[MemberData(nameof(EstrategiasYHilos))]
public void Aggregate_ProduceLosMismosTotalesQueElBaseline(IAggregationStrategy estrategia, int hilos)
{
    var path = Path.Combine(Path.GetTempPath(), $"ventas_agg_test_{Guid.NewGuid():N}.csv");
    try
    {
        SalesDataGenerator.GenerateCsv(path, 5_000);
        var registros = SalesCsvReader.ReadAll(path);

        var esperado = new SequentialAggregator().Aggregate(registros, 1);
        var obtenido = estrategia.Aggregate(registros, hilos);

        Assert.True(
            esperado.TotalesCoinciden(obtenido),
            $"{estrategia.Nombre} con {hilos} hilos no coincide con el baseline secuencial.");
    }
    finally
    {
        File.Delete(path);
    }
}
```

Es una prueba parametrizada: se ejecuta con cada combinación de las 8 estrategias paralelas y 4 configuraciones de hilos (1, 2, 4 y 8) — 32 casos de una sola definición. Cada uno compara contra el agregador secuencial, que por ser de un solo hilo no puede tener condiciones de carrera y sirve de fuente de verdad.

Generación de los 32 casos

```
public static IEnumerable<object[]> EstrategiasYHilos()
{
    foreach (var estrategia in EstrategiasParalelas)
        foreach (var hilos in new[] { 1, 2, 4, 8 })
            yield return [estrategia, hilos];
}
```

9.2 El resto de la suite

Archivo	Qué verifica
AggregationStrategyTests	Que las 8 estrategias paralelas coincidan con el baseline, y que los diagnósticos se poblen (o no) según corresponda.
AggregationResultMergeTests	Que MergeFrom sume correctamente claves existentes y agregue las nuevas.
ContiguousRangePartitionerTests	Que las particiones cubran todo el rango, sin huecos ni solapamientos.
RoundRobinPartitionerTests	Lo mismo para el reparto alternado, incluyendo el reparto del resto.
SalesCsvRoundTripTests	Que lo que escribe el generador sea exactamente lo que lee el parser.

La prueba de particionado que más valor aporta

Verificar que las particiones no se solapen es lo que descarta el error más difícil de detectar en paralelismo: si dos hilos procesaran la misma fila, el total saldría inflado, y en una corrida de cinco millones de filas nadie lo notaría a simple vista.

10. Cómo se mide, y cómo se midió mal

La capa que no aparece en el código pero decide si los números significan algo.

Esta sección documenta un error propio. La primera versión de este proyecto reportaba eficiencias **imposibles**: hasta 187 % con 2 hilos y 172 % con un solo hilo. Una estrategia paralela ejecutada con 1 hilo hace el mismo trabajo que la secuencial más el overhead de particionar y fusionar: nunca puede superar el 100 %. Esa cifra imposible fue la pista de que el problema estaba en el instrumento, no en lo medido.

La causa: la primera medición del proceso

Las fórmulas siempre fueron correctas. El problema era el valor de `T1`: se tomaba de **una única ejecución** que además era **la primera del proceso**. .NET compila por niveles —*tiered compilation*—: el código caliente se promueve a su versión optimizada recién después de varias ejecuciones. Como las nueve estrategias comparten el mismo método de acumulación, la primera de la tabla, el baseline secuencial, corría con código aún sin optimizar y quedaba penalizada, inflando el speedup de todas las demás.

Midiendo el agregador secuencial diez veces seguidas, sobre el mismo dato y el mismo código:

```
La primera es 50 % más lenta; el resto es estable

#1  0.1622s  <-- la unica que media la version anterior
#2  0.1082s
#3  0.1133s
#4  0.1135s
...
#10 0.1092s
```

Las correcciones

Qué	Dónde
Calentamiento global: se ejecutan todas las estrategias una vez antes de medir cualquiera, para que el orden dentro de la tabla no influya.	Cli/Program.cs
Repeticiones: cada medición descarta un calentamiento y reporta el mejor de tres tiempos. Se usa el mínimo y no el promedio porque el ruido de una máquina de uso general solo puede restar velocidad, nunca sumarla.	Metrics/BenchmarkRunner.cs
Baseline correcto en el barrido: antes se tomaba como referencia el primer valor de <code>--hilos</code> . Con <code>--hilos 2,4,8</code> la tabla mostraba <code>hilos=2 speedup=1.00 eficiencia=50 %</code> bajo un encabezado que decía "respecto a 1 hilo".	Cli/Program.cs

Tras la corrección, las cinco estrategias sin contención miden entre **0.93 y 1.02 de speedup con 1 hilo** — exactamente lo que predice la teoría— frente al rango de 1.32 a 1.72 que se reportaba antes.

Una conclusión retirada

La versión anterior afirmaba que activar **Server GC** mejoraba la eficiencia entre 6 y 32 puntos porcentuales, y lo presentaba como prueba de que el recolector de basura era un punto de sincronización oculto. **Esa conclusión**

era falsa: comparaba mediciones tomadas con la metodología defectuosa. Repitiendo la comparación con el método corregido y ejecutando ambas configuraciones una tras otra:

Estrategia (8 hilos)	Workstation GC	Server GC
Chunking dinámico	42 %	41 %
Reducción en árbol	43 %	38 %
Acumuladores locales	39 %	35 %
Grano grueso	39 %	40 %
Round-robin	29 %	29 %

Las diferencias son menores que la variación entre corridas de una misma configuración: en este banco de pruebas Server GC **no produce una mejora medible**. Es coherente con el resto de los datos —los acumuladores tienen 8 y 12 claves, así que la presión sobre el heap es mínima y el recolector casi no interviene—. La opción quedó activada por ser la recomendada para cargas paralelas, pero no se le atribuye ninguna mejora medida aquí.

Resumen: dónde vive la sincronización

Nivel	Mecanismo	¿Se midió su efecto?
Aplicación	Locks explícitos sobre los acumuladores	Sí — de 6 % a 41 % de eficiencia
Biblioteca	Candados internos de ConcurrentDictionary , particionado de PLINQ	No es medible desde afuera
Runtime	Pausas del recolector de basura	Sí — sin efecto detectable
Hardware	Núcleos físicos, caché, ancho de banda de memoria	Es el techo real: ~40 % con 8 hilos

La conclusión que atraviesa todas las capas

El proyecto elimina la sincronización por fila y con eso pasa de 6 % a 41 % de eficiencia. A partir de ahí, ni el particionado fino ni la configuración del runtime mueven la aguja: el techo lo pone el hardware —4 núcleos físicos y, en datasets grandes, el ancho de banda de memoria—, y eso ya no se programa. Averiguarlo exigió, primero, arreglar el instrumento de medición.